ProofLine

Verifiable On-Device Visual Quality Inspection

Stage I Proposal — ASUS UGen AI League Innovation Hackathon

Theme	Workplace AI (The Workplace)
Battlefield	Battlefield Lightning — Lightweight Computing Track
Finals track	Global Online
Team	Slid Phi Labs
Code	https://github.com/ceedot-rock/proofline
Proposal date	October 9, 2026

Deadline: October 14, 2026, 05:00 EDT

1. The problem — scenario

Picture a contract manufacturer shipping stamped metal brackets. A customer rejects a shipment: "defective parts in lot 44." The line ran an edge-AI camera that passed every part. Now the customer's quality engineer, the plant's QA (quality assurance) manager, and the contract lawyers all ask the same question: what did the model actually see, and what did it decide, for each part?

Today, nobody can answer with proof. The inspection verdict lives in a log line, a dashboard count, maybe a CSV (comma-separated values) someone exported by hand. Logs can be edited. Timestamps can drift. Which model version ran at 2:14 a.m. is a guess. So the dispute settles the expensive way: re-inspection, chargebacks, and a recall scoped by calendar instead of by evidence — the whole week gets quarantined because nobody can prove which hours were clean.

Edge AI made inspection fast and cheap, but it left a hole: **the verdict is not evidence.** ProofLine closes that hole.

2. The solution

ProofLine is verifiable on-device visual quality inspection. Each inspection — one image in — produces a verdict: pass or fail, which model ran, and a timestamp. That verdict is written into a tamper-evident journal: every entry hash-chained to the one before it, each entry digitally signed (Ed25519 signature scheme), and the whole journal replayable — any auditor can re-run the chain and re-verify every signature, offline, on the device itself.

What this proves, precisely:

- that a specific image produced a specific verdict, from a specific model version, at a specific time;
- that the record has not been altered since — any edit breaks the hash chain and invalidates signatures;
- that the record cannot be silently reordered or backdated — the chain commits the order.

What it does **not** prove: it does not prove the model was right. Correctness is a model-quality question, settled by validation. ProofLine proves the **provenance** of the decision — which is what disputes and audits actually argue about.

3. Commercial feasibility

Who pays: QA managers and contract manufacturers who absorb dispute costs, and automotive and medical-device suppliers who live under audit regimes. (Example: IATF 16949, the automotive quality-management standard, requires documented evidence of inspection.)

Value, in plain terms:

- **Disputes** get decided by replaying the journal instead of re-inspecting shipments.
- **Compliance audits** get a single artifact: the signed journal, verifiable offline with no vendor in the loop.
- **Recalls** get scoped to the exact serials and hours the journal shows as failed, instead of the whole lot.

Business model (to be validated in Stage II): per-production-line subscription, priced against dispute-cost savings; the journal format is open so customers are never locked into our viewer.

Honest scope: Stage I validates the mechanism — journal, signatures, replay. Pricing, pilots, and on-device model work are Stage II.

4. Innovation — what is new

- **The journal is the product, not the model.** Existing vision systems treat logging as an afterthought. ProofLine makes the tamper-evident record a first-class output of the inspection pipeline.
- **Edge-native verifiability.** Hash chain plus Ed25519 signing plus offline replay runs on the device itself — no cloud round-trip, no trusted third party at inspection time.
- **New application, honest foundation.** ProofLine is a new application, built for this hackathon. It builds on the public PmCp runtime — Persistent Memory Core Processing, Apache-2.0, at github.com/ceedot-rock/pmcp — which provides the extent manager, the crash-safe journal, and self-contained SHA-256 (Secure Hash Algorithm, 256-bit). PmCp is the foundation, not the entry. The entry is the inspection-verdict application: the verdict schema, the signing layer, and the inspect-to-verify pipeline, all of which live in the ProofLine repo.

5. Intended tools and methods

BUILT — Stage I (this proposal):

- Append-only, crash-safe inspection journal with hash-chained entries.
- Ed25519 sign and verify per entry, implemented in the ProofLine repo.
- Working inspect-to-verify pipeline: `proofline inspect <image>` writes a signed verdict; `proofline verify` replays the chain and checks every signature.
- Demo on synthetic images with a labeled heuristic classifier.

PLANNED — Stage II:

- A real defect-detection model compiled for the Hailo-10H on the ASUS UGen300.
- Measured on-device throughput, journaling overhead, and verification time.

NOT claimed: no integration with any external attestation service exists. The signing layer is ProofLine's own.

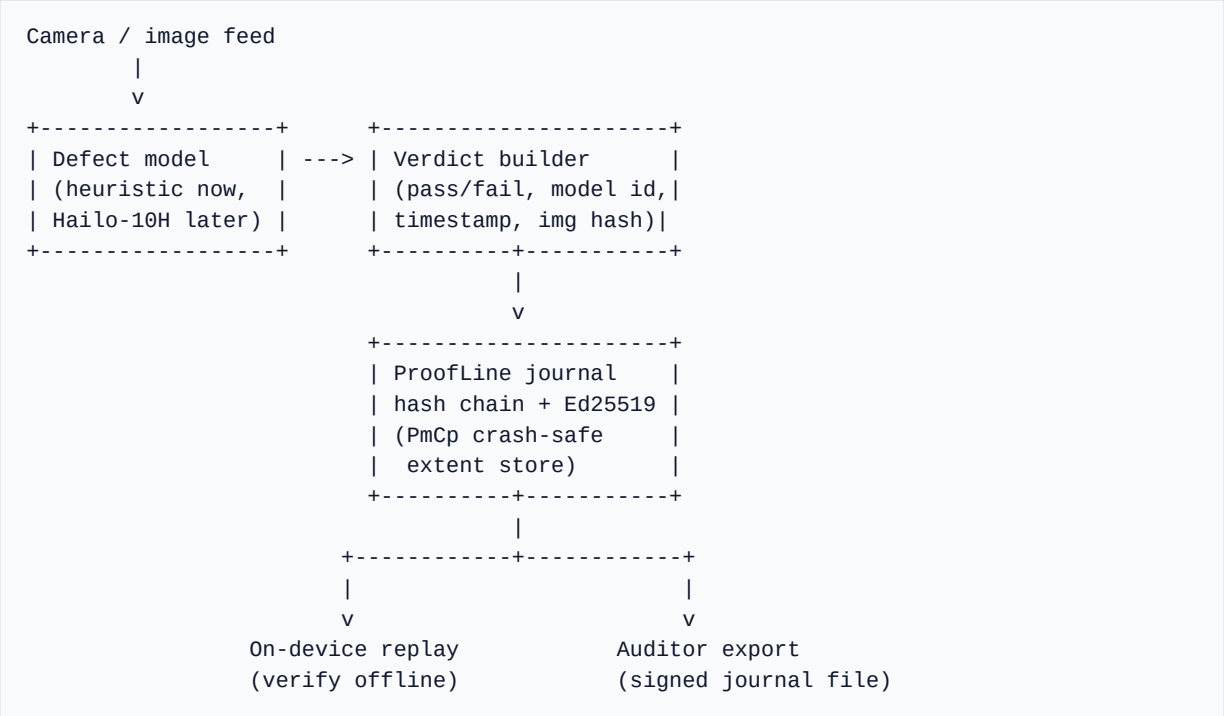
Classifier honesty, stated plainly: the Stage I demo classifier is a **labeled heuristic over synthetic images** — it demonstrates the pipeline end to end, not production accuracy.

6. Hardware and software architecture

Hardware.

- Stage I runs on a laptop. There is no UGen300 in the Stage I build — stated plainly. All journal, signing, and verification logic is hardware-agnostic.
- Stage II target: ASUS UGen300 AI Accelerator — Hailo-10H chip, up to 40 TOPS (INT4) (40 trillion operations per second, 4-bit integer arithmetic), 8 GB of dedicated memory. Form factor, M.2 or USB: **TBD** — the owner selects it at submission, and it cannot be changed later.

Software. Five layers, each with one job:



Capture feeds inference; inference feeds verdict assembly; every verdict is appended to the journal (hash-chained, Ed25519-signed) on the PmCp crash-safe extent store. Verification is either local replay or an exported signed journal file an auditor checks independently. CLI (command-line interface): `proofline inspect`, `proofline verify`, `proofline export`.

7. Verdict record schema

Each journal entry carries exactly what an auditor needs and nothing more:

```
{
  "seq":          42,
  "prev_hash":    "9f2c...a1",
  "image_sha256":"71bd...e0",
  "model_id":     "heuristic-v0.1",
  "verdict":      "FAIL",
  "confidence":   0.93,
  "timestamp_utc":"2026-10-09T14:02:11Z",
  "signer_pubkey":"ed25519:3fa4...c9",
  "signature":    "5d20...77"
}
```

`prev_hash` chains the entry to its predecessor; `image_sha256` binds the verdict to the exact input image; `model_id` pins which model decided; the Ed25519 `signature` covers the whole entry. Replay recomputes every hash and re-checks every signature — one broken link fails the verification.

8. Trust model — what "tamper-evident" means

- **Hash chain:** entry N commits to entry N-1. Altering any entry breaks every later hash.
- **Ed25519 signature per entry:** proves the entry was written by the holder of the device key.
- **Offline replay:** `proofline verify` recomputes hashes and checks signatures with no network access.

Limits, stated up front:

- ProofLine protects the record **after** it is written. It does not secure the camera feed itself.
- It does not prove the model was correct — only what the model decided.
- Key custody is the operator's responsibility. Stage II includes a device key-provisioning plan so a stolen device key cannot silently mint history.

9. Stage II plan

1. Bring up the UGen300 (form factor TBD at submission).
2. Compile a real defect-detection model — a small object-detection CNN (convolutional neural network) — for the Hailo-10H via the Hailo toolchain.
3. Measure: sustained inspections per second at line rate, journal append latency, verification time per 10,000 entries.
4. Run the end-to-end demo on real defect imagery and publish the measured numbers.

Stage II exit criteria: a working on-device demo with measured throughput and a verification-time curve — no unmeasured claims.

10. Risks and mitigations

- **Model accuracy vs. record integrity.** Separate problems, scoped separately. Stage II uses a standard defect dataset and model; ProofLine's claim is provenance, not accuracy.
- **Key management on shared devices.** Per-device keys; Stage II provisioning plan; key rotation invalidates nothing historical because old signatures still verify against the old public key.
- **Journal growth on long runs.** The PmCp extent manager compacts storage; verification streams, so memory stays flat regardless of journal length.
- **Form-factor lock-in.** M.2 vs. USB is chosen once at submission. The journal and signing layers are form-factor-agnostic, so the choice affects only the Stage II bring-up path.

11. Expected outcomes — Stage I

- An open repository with a working inspect-to-verify pipeline and a synthetic-image demo.
- A documented verdict schema and an explicit trust model, limits included.
- A Stage II plan with measurable targets on the UGen300.
- A commercial story grounded in dispute, audit, and recall economics — the 35% the judging weights most.

12. References

- ProofLine (this entry): https://github.com/ceedot-rock/proofline
- PmCp — Persistent Memory Core Processing (Apache-2.0): https://github.com/ceedot-rock/pmcp
- Hailo-10H AI accelerator and Hailo AI toolchain: https://hailo.ai (Stage II compile target)
- Bernstein, Duif, Lange, Schwabe, Yang, "High-speed high-security signatures," Journal of Cryptographic Engineering, 2012 (Ed25519)
- IATF 16949:2016 — quality management systems for automotive production (documented inspection evidence)
- ASUS UGen AI League Innovation Hackathon — Battlefield Lightning, Lightweight Computing Track (Stage I rules)

Appendix A — repository layout

```
proofline/
  src/     journal, hash chain, Ed25519 signing layer
  cli/     proofline inspect / verify / export
  demo/    synthetic image generator + labeled heuristic classifier
  docs/    verdict schema, trust model, Stage II plan
```

Appendix B — demo transcript (illustrative)

```
$ proofline inspect demo/part_0042.png
verdict=FAIL model=heuristic-v0.1 confidence=0.93 seq=42 signature=ok

$ proofline verify
420/420 entries verified, chain intact
```